

Code Smells y Refactorings

Code smell: señal en el código de que existe un problema de diseño profundo. No es un error, pero indica que el código será difícil de mantener, entender o modificar.

Refactoring: transformación del código que mejora su estructura **sin cambiar su comportamiento observable**.

Cómo usar esta guía

1. **Identificá** el smell con su nombre exacto (en el parcial vale el nombre en inglés o español).
2. **Justificá** con evidencia del código ("el método hace X, Y y Z a la vez...").
3. **Resolvé** con uno o más refactorings de la tabla.

Glosario: Bad Smells (inglés → español)

En parcial vale el nombre en inglés o el español. Usá el que te quede más cómodo, pero mantenelo consistente dentro de la misma respuesta.

Inglés	Español	Traducción literal / sinónimo frecuente
Alternative Classes with Different Interfaces	Clases alternativas con interfaces distintas	—
Comments	Comentarios (excesivos)	—
Data Class	Clase de datos	—
Data Clumps	Racimos de datos	Grupos de datos, atados de datos
Divergent Change	Cambio divergente	—
Duplicated Code	Código duplicado	—
Feature Envy	Envidia de atributos	Envidia de datos, envidia de clases
Incomplete Library Class	Clase de librería incompleta	—
Inappropriate Intimacy	Intimidad inapropiada	Acoplamiento excesivo entre clases
Large Class	Clase grande	—
Lazy Class (Freeloader)	Clase perezosa	—
Long Method	Método largo	—
Long Parameter List	Lista larga de parámetros	—

Inglés	Español	Traducción literal / sinónimo frecuente
Message Chains	Cadenas de mensajes	Cadenas de delegación
Middle Man	Intermediario	Hombre del medio
Parallel Inheritance Hierarchies	Jerarquías de herencia paralelas	—
Primitive Obsession	Obsesión por tipos primitivos	Obsesión por primitivos
Refused Bequest	Herencia rechazada	Legado rechazado
Shotgun Surgery	Cirugía de escopeta	—
Speculative Generality	Generalidad especulativa	—
Switch Statement	Sentencia switch	Condicional por código de tipo
Temporary Field	Campo temporal	Atributo temporal

Glosario: Refactorings (inglés → español)

Inglés	Español
Change Bidirectional Association to Unidirectional	Cambiar asociación bidireccional a unidireccional
Collapse Hierarchy	Colapsar jerarquía
Decompose Conditional	Descomponer condicional
Encapsulate Collection	Encapsular colección
Encapsulate Field	Encapsular atributo
Extract Class	Extraer clase
Extract Interface	Extraer interfaz
Extract Method	Extraer método
Extract Subclass	Extraer subclase
Extract Superclass	Extraer superclase
Form Template Method	Formar método plantilla
Hide Delegate	Ocultar delegado
Hide Method	Ocultar método
Inline Class	Clase en línea (Inline Class)
Inline Method	Método en línea (Inline Method)
Introduce Assertion	Introducir aserción

Inglés	Español
Introduce Foreign Method	Introducir método externo
Introduce Local Extension	Introducir extensión local
Introduce Null Object	Introducir objeto nulo
Introduce Parameter Object	Introducir objeto parámetro
Move Field	Mover atributo
Move Method	Mover método
Pull Up Field	Subir atributo
Pull Up Method	Subir método
Push Down Field	Bajar atributo
Push Down Method	Bajar método
Remove Middle Man	Eliminar intermediario
Remove Parameter	Eliminar parámetro
Remove Setting Method	Eliminar método setter
Rename Method	Renombrar método
Rename Variable	Renombrar variable
Replace Array with Object	Reemplazar arreglo con objeto
Replace Conditional with Polymorphism	Reemplazar condicional con polimorfismo
Replace Data Value with Object	Reemplazar valor de dato con objeto
Replace Delegation with Inheritance	Reemplazar delegación con herencia
Replace Inheritance with Delegation	Reemplazar herencia con delegación
Replace Loop with Pipeline	Reemplazar bucle con pipeline
Replace Magic Number with Symbolic Constant	Reemplazar número mágico con constante simbólica
Replace Method with Method Object	Reemplazar método con objeto método
Replace Parameter with Explicit Methods	Reemplazar parámetro con métodos explícitos
Replace Parameter with Method	Reemplazar parámetro con método
Replace Primitive with Object	Reemplazar primitivo con objeto
Replace Temp with Query	Reemplazar temporal con consulta
Replace Type Code with Class	Reemplazar código de tipo con clase
Replace Type Code with State/Strategy	Reemplazar código de tipo con State/Strategy

Inglés	Español
Replace Type Code with Subclasses	Reemplazar código de tipo con subclases
Substitute Algorithm	Sustituir algoritmo

Truco para recordar los "Replace X with Y": siempre se leen igual en ambos idiomas — *reemplazar lo que está mal (X) por la solución (Y)*. Si te olvidás la traducción exacta en el parcial, tradúcelo literal: se entiende igual.

Tabla de Smells y Refactorings

Smell	Cómo detectarlo	Refactorings
Alternative Classes with Different Interfaces	Dos clases hacen lo mismo con nombres/métodos distintos	Rename Method Move Method Extract Superclass
Comments	El comentario explica <i>qué</i> hace el código en lugar de <i>por qué</i> ; el nombre no alcanza	Rename Method Extract Method Introduce Assertion
Data Class	Clase con solo atributos y getters/setters; otros clases hacen la lógica con sus datos	Move Method Encapsulate Field Encapsulate Collection Remove Setting Method Extract Method Hide Method
Data Clumps	Parámetros o variables que siempre viajan juntos (ej. <code>fechaInicio</code> , <code>fechaFin</code>)	Extract Class Preserve Whole Object Introduce Parameter Object
Divergent Change	La misma clase se modifica por motivos completamente distintos según qué funcionalidad cambie	Extract Class
Duplicated Code	Dos o más fragmentos de código idénticos o casi idénticos	Extract Method Pull Up Field Form Template Method Substitute Algorithm Extract Class Introduce Null Object
Feature Envy	Un método usa más datos de otra clase que de la propia	Extract Method Move Method Move Field

Smell	Cómo detectarlo	Refactorings
Lazy Class (Freeloader)	Clase que no hace lo suficiente para justificar su existencia	Collapse Hierarchy Inline Class
Inappropriate Intimacy	Dos clases conocen demasiados detalles internos la una de la otra	Move Method Move Field Change Bidirectional Association to Unidirectional Association Extract Class Hide Delegate Replace Inheritance with Delegation
Incomplete Library Class	La librería no tiene lo que necesitás y no podés modificarla	Introduce Foreign Method Introduce Local Extension
Large Class	Una clase con demasiadas responsabilidades/atributos/métodos	Extract Class Extract Subclass Extract Interface Replace Data Value with Object
Long Method	Método largo; las variables temporales y los loops son "zonas calientes"	Extract Method Introduce Parameter Object Decompose Conditional Preserve Whole Object Replace Method with Method Object Replace Temp with Query
Long Parameter List	Firmas con muchos parámetros, varios de los cuales se repiten en otros métodos	Replace Parameter with Method Introduce Parameter Object Preserve Whole Object
Message Chains	Cadenas tipo <code>a.getB().getC().getD().foo()</code>	Hide Delegate Extract Method Move Method
Middle Man	Una clase que solo delega la mayoría de sus operaciones a otra	Remove Middle Man Inline Method Replace Delegation with Inheritance

Smell	Cómo detectarlo	Refactorings
Parallel Inheritance Hierarchies	Cada vez que creás una subclase de una, tenés que crear una de la otra	Move Method Move Field
Primitive Obsession	Uso de primitivos (<code>int</code> , <code>String</code>) para conceptos con dominio propio (código, moneda, rango)	Replace Data Value with Object Introduce Parameter Object Extract Class Replace Type Code with Class Replace Code with State/Strategy Replace Type Code with Subclasses Replace Array with Object
Refused Bequest	Una subclase que no usa nada o casi nada de lo que hereda	Push Down Field Push Down Method Replace Inheritance with Delegation
Shotgun Surgery	Un cambio pequeño obliga a tocar muchas clases distintas	Move Method Move Field Inline Class
Speculative Generality	Código muerto: métodos/parámetros genéricos "por si las moscas"	Collapse Hierarchy Rename Method Remove Parameter Inline Class
Switch Statement	<code>switch/if-else</code> que se repite cada vez que aparece un nuevo tipo de dato	Extract Method Move Method Replace Type Code with Subclasses Replace Type Code with State/Strategy Replace Conditional with Polymorphism Replace Parameter with Explicit Methods Introduce Null Object
Temporary Field	Atributos que solo se usan dentro de ciertos métodos o ramas; el resto del tiempo están vacíos	Extract Class Introduce Null Object

Smells clave: cómo justificarlos en el parcial

Smell	Frase de justificación típica
Long Method	"El método hace A, B y C a la vez; se hace difícil ver de un vistazo qué hace" → <i>Extract Method</i>
Feature Envy	"Este método se pasa la vida usando los datos de OtraClase y casi no usa los suyos" → <i>Move Method / Extract Method</i>
Data Class	"La clase solo guarda datos y no tiene comportamiento; la lógica vive en otras clases" → <i>Move Method</i>
Duplicated Code	"Este bloque aparece copiado en dos lugares; si hay que corregirlo hay que corregirlo en los dos" → <i>Extract Method</i>
Primitive Obsession	"Usa un String para representar un concepto del dominio, perdiendo validación y expresividad" → <i>Replace Primitive with Object</i>
Data Clumps	"Estos 3 parámetros siempre viajan juntos" → <i>Introduce Parameter Object</i>

Smells clave: ejemplos cortitos

Long Method

"El método hace A, B y C a la vez; se hace difícil ver de un vistazo qué hace" → *Extract Method*

```
// ANTES: calcula, imprime y acumula todo junto
for (Empleado e : empleados) {
    double bruto = e.getSueldoBasico() + e.getHorasExtra() * 450 + ...; // 5
    líneas de cuenta
    System.out.println("Empleado: " + e.getNombre() + " - Neto: " + neto);
    totalGeneral += neto;
}

// DESPUÉS: cada línea se entiende de un vistazo
for (Empleado e : empleados) {
    imprimirRecibo(e);
    totalGeneral += calcularNeto(e);
}
```

Feature Envy

"Este método se pasa la vida usando los datos de **OtraClase** y casi no usa los suyos" → *Move Method / Extract Method*

```
// ANTES: ImpresorFacturas vive mirando los atributos de Direccion
public String generarEncabezado(Factura f) {
    return "..." + f.getCliente().getDireccion().getCalle()
        + " " + f.getCliente().getDireccion().getCiudad();
}
```

```
// DESPUÉS: el formateo vive en Direccion, que es dueña de esos datos
public String generarEncabezado(Factura f) {
    return "... " + f.getDireccionFormateada();
}
```

Data Class

"La clase solo guarda datos y no tiene comportamiento; la lógica vive en otras clases" → *Move Method*

```
// ANTES: Producto solo tiene getters/setters y Carrito le hace la cuenta
class Carrito {
    double total() { return items.stream()
        .mapToDouble(i -> i.getProducto().getPrecio() * i.getCantidad()).sum(); }
}

// DESPUÉS: cada objeto calcula con sus propios datos
class ItemCarrito {
    double calcularTotal() { return producto.getPrecio() * this.cantidad; }
}
```

Duplicated Code

"Este bloque aparece copiado en dos lugares; si hay que corregirlo hay que corregirlo en los dos" → *Extract Method*

```
// ANTES: la fórmula del bruto está copiada en 3 métodos
double bruto = e.getSueldoBasico() + e.getHorasExtra() * 450 + e.getAntiguedad() * 200;
double descuento = bruto * 0.13;

// DESPUÉS: una sola fórmula, un solo lugar que tocar
double neto = calcularNeto(e); // (o Empleado.getSueldoNeto() con Move Method)
```

Primitive Obsession

"Usa un `String` para representar un concepto del dominio, perdiendo validación y expresividad" → *Replace Primitive with Object*

```
// ANTES: cualquier string pasa, y == compara referencias (bug)
if (tipoHabitacion == "simple") { total = total * 1.0; }

// DESPUÉS: solo existen valores válidos, y la lógica vive en el objeto
total *= tipoHabitacion.getFactor(); // Simple(1.0), Doble(1.5), Suite(2.2)
```

Data Clumps

"Estos 3 parámetros siempre viajan juntos" → *Introduce Parameter Object*

```
// ANTES: las fechas viajan siempre pegadas
public boolean disponibilidad(String fechaInicio, String fechaFin, String tipo) {
    ... }
public double calcularPrecio(String fechaInicio, String fechaFin, ...) { ... }

// DESPUÉS: un objeto que encarna el concepto "estadía"
public boolean disponibilidad(RangoFechas estadia, TipoHabitacion tipo) { ... }
```

Switch Statement (bonus)

"Cada vez que aparece un tipo nuevo hay que tocar este condicional" → *Replace Conditional with Polymorphism*

```
// ANTES: crece con cada tipo nuevo
if (tipo == "nacional") { auxc = duracion * 3 + iva; }
else if (tipo == "internacional") { auxc = duracion * 150 + iva + 50; }

// DESPUÉS: agregar un tipo = agregar una clase (Open/Closed)
auxc = llamada.calcularCosto();
```

Middle Man (bonus)

"Esta clase solo delega, no agrega valor" → *Remove Middle Man / Inline Class*

```
// ANTES: Catalogo es un pasamanos
public List<Libro> buscarPorTitulo(String t) { return db.buscarPorTitulo(t); }
// ... 5 métodos más idénticos ...

// DESPUÉS: el cliente habla directo con db (o Catalogo gana lógica propia)
```

Lazy Class vs Data Class (bonus: ¡ojo que se confunden!)

```
class Avioneta extends Vehiculo { }           // LAZY CLASS: no hace NADA → Inline
Class / Collapse Hierarchy
class Producto { String nombre; double precio; /* solo getters/setters */ }
                                           // DATA CLASS: guarda datos y otros

hacen la lógica → Move Method
```

Temporary Field (bonus)

"Este atributo solo se llena en un método y el resto del tiempo está vacío" → *Extract Class / Introduce Null Object*

```
private String ultimoPrestamoDetalle;    // se setea SOLO en registrar() y se lee
en un método
private LocalDate ultimaFechaDevolucion;
// DESPUÉS: viven en un objeto Prestamo/ResumenPrestamo que existe solo cuando
hace falta
```

Comments (bonus)

"El comentario explica QUÉ hace la línea, no POR QUÉ" → *Rename Method / Replace Magic Number with Symbolic Constant*

```
// ANTES
double descuento = bruto * 0.13;    // le aplico el 13% de descuento por aportes

// DESPUÉS: el código se explica solo, el comentario sobra
double descuento = bruto * PORCENTAJE_APORTES;
```

Refactorings más frecuentes (qué hacen)

Refactoring	Qué hace	Ejemplo
Rename Method	Pone un nombre que diga qué hace	<code>lmtCrdt()</code> → <code>getLimiteCredito()</code>
Extract Method	Saca un fragmento a un método nuevo con nombre propio	Separar el cálculo del promedio de la impresión
Move Method	Mueve un método a la clase donde más datos usa	<code>Persona.participaEnProyecto()</code> → <code>Proyecto.participa()</code>
Encapsulate Field	Hace privado el atributo y expone acceso controlado	<code>+id</code> → <code>-id + getId()</code>
Introduce Parameter Object	Reemplaza parámetros que viajan juntos por un objeto	<code>(f1, f2)</code> → <code>(RangoFechas rango)</code>
Replace Temp with Query	Elimina variables intermedias usando directamente el método	<code>double p = calc(); fmt(p)</code> → <code>fmt(calc())</code>
Preserve Whole Object	Pasa el objeto entero en vez de extraer sus partes	<code>f(inicio, fin)</code> → <code>f(rango)</code>
Substitute Algorithm	Reemplaza el algoritmo por uno más simple	Bucles manuales → Streams

Refactoring	Qué hace	Ejemplo
Inline Method	Elimina un método trivial usando su cuerpo directamente	—
Hide Delegate	Encapsula una asociación para no chainear delegaciones	<code>cliente.getDepartamento().getJefe()</code> → <code>cliente.getJefe()</code>

Reglas rápidas de detección

- **Método largo + variables temporales** → Long Method
- **Muchos parámetros que se repiten** → Long Parameter List / Data Clumps
- **Usa datos de otra clase > que los propios** → Feature Envy
- **switch que crece con cada tipo nuevo** → Switch Statement
- **Clase con solo getters/setters** → Data Class
- **Clase que solo delega** → Middle Man
- **Atributo que solo se llena en un método** → Temporary Field
- **Código copiado y pegado** → Duplicated Code